

Министерство образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №5
по курсу «Разработка кроссплатформенных приложений»
на тему «Многопоточное вычисление определенного интеграла»

Выполнили студенты группы

23ВВИ1:

Видяев А.А.

Жиганов Н.Д.

Приняли:

к.т.н. доцент Юрова О.В.

к.т.н. доцент Карамышева Н.С.

Пенза 2026

Цель работы

Изучить основы многопоточного программирования в Java и модифицировать приложение из предыдущей лабораторной работы так, чтобы вычисление определенного интеграла выполнялось параллельно в нескольких потоках без блокировки графического интерфейса.

Задание на лабораторную работу

Модифицировать программу из лабораторной работы №4. Реализовать многопоточное вычисление определенного интеграла функции $\sqrt{x}$. Для варианта 6 необходимо использовать 6 дополнительных потоков. Так как номер варианта больше 5, потоки должны быть реализованы на основе интерфейса `Runnable`. Пользовательский интерфейс должен оставаться работоспособным в процессе вычислений.

Ход работы

1. Создание класса `IntegralTask`. Для реализации требования о применении `Runnable` был создан отдельный класс `IntegralTask`, который реализует интерфейс `Runnable`. Каждый объект данного класса получает начало подинтервала, конец подинтервала и шаг интегрирования. В методе `run()` выполняется вычисление частичной суммы методом прямоугольников для функции $\sqrt{x}$. Результат сохраняется во внутреннем поле `partialResult` и затем считывается после завершения потока.

```

public class IntegralTask implements Runnable {
    private final double start;
    private final double end;
    private final double step;
    private double partialResult;

    public IntegralTask(double start, double end, double step) {
        this.start = start;
        this.end = end;
        this.step = step;
        this.partialResult = 0.0;
    }

    @Override
    public void run() {
        double x = start;

        while (x < end) {
            double nextX = Math.min(x + step, end);
            partialResult += Math.sqrt(x) * (nextX - x);
            x = nextX;
        }

        public double getPartialResult() {
            return partialResult;
        }
    }
}

```

2. Модификация класса RecIntegral. В классе RecIntegral была добавлена константа `THREAD_COUNT = 6`, отражающая номер варианта. Также был реализован метод `integrateSqrtMultiThreaded()`, который делит общий интервал интегрирования на 6 частей, создает для каждой части объект `IntegralTask`, запускает соответствующие потоки и ожидает их завершения через метод `join()`. После завершения всех потоков выполняется суммирование частичных результатов.

```

public double integrateSqrtMultiThreaded(double lp, double hp, double step) throws InterruptedException {
    if (lp < 0 || hp < 0) {
        throw new IllegalArgumentException("Границы интегрирования не могут быть отрицательными.");
    }
    if (step <= 0) {
        throw new IllegalArgumentException("Шаг должен быть положительным.");
    }
    if (lp > hp) {
        throw new IllegalArgumentException("Нижняя граница не может быть больше верхней.");
    }

    double intervalLength = hp - lp;
    double partLength = intervalLength / THREAD_COUNT;

    IntegralTask[] tasks = new IntegralTask[THREAD_COUNT];
    Thread[] threads = new Thread[THREAD_COUNT];

    double start = lp;

    for (int i = 0; i < THREAD_COUNT; i++) {
        double end = (i == THREAD_COUNT - 1) ? hp : start + partLength;

        tasks[i] = new IntegralTask(start, end, step);
        threads[i] = new Thread(tasks[i]);
        threads[i].start();

        start = end;
    }

    double total = 0.0;

    for (int i = 0; i < THREAD_COUNT; i++) {
        threads[i].join();
        total += tasks[i].getPartialResult();
    }

    return total;
}

```

3. Сохранение логики проверки данных. Все проверки, добавленные в предыдущих лабораторных работах, были сохранены: контроль диапазона значений lower, upper и step, запрет отрицательных границ интегрирования, проверка корректности шага и генерация пользовательского исключения `InvalidRangeException` при ошибках ввода.

```

public class InvalidRangeException extends Exception {
    public InvalidRangeException(String message) {
        super(message);
    }
}

```

4. Изменение обработчика кнопки «Вычислить». В классе `Ib1UI` обработчик `jButtonCalculateActionPerformed(...)` был модифицирован таким образом, чтобы запуск вычисления происходил не в потоке обработки событий `Swing`, а в отдельном фоновом потоке. Это исключает зависание окна во время выполнения длительных операций.

```

private void jButtonCalculateActionPerformed(java.awt.event.ActionEvent evt) {
int selected = jTable2.getSelectedRow();
if (selected == -1) {
    JOptionPane.showMessageDialog(this,
        "Выберите строку для вычисления.",
        "Предупреждение", JOptionPane.WARNING_MESSAGE);
    return;
}

DefaultTableModel model = (DefaultTableModel) jTable2.getModel();

try {
    double lower = Double.parseDouble(model.getValueAt(selected, 0).toString());
    double upper = Double.parseDouble(model.getValueAt(selected, 1).toString());
    double step = Double.parseDouble(model.getValueAt(selected, 2).toString());

    RecIntegral rec = records.get(selected);
    rec.setLower(lower);
    rec.setUpper(upper);
    rec.setStep(step);

    jButtonCalculate.setEnabled(false);

    new Thread(() -> {
        try {
            double result = calculateDistributed(lower, upper, step);
            rec.setResult(result);

            javax.swing.SwingUtilities.invokeLater(() -> {
                model.setValueAt(String.format("%.4f", result), selected, 3);
                jButtonCalculate.setEnabled(true);

                JOptionPane.showMessageDialog(this,
                    "Распределённое вычисление завершено.",
                    "Успех",
                    JOptionPane.INFORMATION_MESSAGE);
            });
        }
    });
}

```

```

    } catch (Exception e) {
        javax.swing.SwingUtilities.invokeLater(() -> {
            jButtonCalculate.setEnabled(true);
            JOptionPane.showMessageDialog(this,
                "Ошибка вычисления: " + e.getMessage(),
                "Ошибка",
                JOptionPane.ERROR_MESSAGE);
        });
    }
}).start();

} catch (NumberFormatException | NullPointerException e) {
    JOptionPane.showMessageDialog(this,
        "Ошибка в данных строки.",
        "Ошибка", JOptionPane.ERROR_MESSAGE);
} catch (InvalidRangeException ex) {
    JOptionPane.showMessageDialog(this,
        ex.getMessage(),
        "Ошибка диапазона", JOptionPane.WARNING_MESSAGE);
}
}

```

Вывод

В ходе лабораторной работы были изучены основные средства многопоточного программирования в Java. Программа из предыдущей лабораторной работы была успешно модифицирована: вычисление определенного интеграла функции $\sqrt{x}$ реализовано в 6 дополнительных потоках на основе интерфейса `Runnable`. Также была обеспечена корректная работа графического интерфейса и сохранена система проверки входных данных. В результате приложение стало выполнять вычисления быстрее и без зависания пользовательского интерфейса.